

Effat University – College of Engineering

Department of Computer Science

CS4074: Big Data Analytics

Lab 5: Programming with Apache Spark

From SparkSession to Execution Plans — End-to-End PySpark Pipeline

Student Name: Hams Aljohani

Student ID: S20106833

Section: _____

Date: _____

Objective: Write, execute, and analyze PySpark programs. Map every operation to the distributed computing architecture, DAG, and memory concepts from the lecture.

Dataset: NYC Yellow Taxi Trip Data — January 2024 (≈ 3 million rows, 51 MB Parquet) + Zone Lookup Table (265 rows, CSV).

Prerequisites: Lab 4 (Spark RDDs), Lecture 5 (PySpark Programming).

Tools: VSCode, PySpark 3.5, Hadoop HDFS, Python 3.8+.

Duration: 3 hours.

Structure: Part 1: Setup (15 min) → Part 2: HDFS (20 min) → Part 3: SparkSession (10 min) → Part 4: Ingestion (15 min) → Part 5: DataFrame API (45 min) → Part 6: Spark SQL (20 min) → Part 7: UDFs (15 min) → Part 8: Partitioning (15 min) → Part 9: Caching (10 min) → Part 10: Output (15 min) → Part 11: Execution Plans (15 min) → Deep Work (30 min).

Part 1

Pre-Lab Setup & VSCode (15 min)

1.1 Pre-Lab Checklist

Run each command before starting. All must pass.

Check	Command	Status
Java 11+ installed	<code>java -version</code>	☐ Pass
HDFS running	<code>jps</code> (NameNode + DataNode)	☐ Pass
PySpark installed	<code>pip show pyspark</code>	☐ Pass
VSCode installed	<code>code -version</code>	☐ Pass
Python 3.8+	<code>python3 -version</code>	☐ Pass
PyArrow (for Pandas UDFs)	<code>pip show pyarrow</code>	☐ Pass

If `jps` does not show **NameNode** and **DataNode**, start HDFS first:

```
1 start-dfs.sh      # wait 20 seconds
2 jps              # verify both processes appear
```

1.2 Create Project Folder and Open in VSCode

► Terminal

```
1 mkdir ~/cs4074_lab5 && cd ~/cs4074_lab5
2 mkdir -p data output src
3 code . # opens VSCode in this folder
```

Your project structure should look like:

```
cs4074_lab5/
|-- data/      <- downloaded files
|-- output/    <- local backup
'-- src/       <- all Python scripts
```

1.3 Install VSCode Extensions

Open the Extensions panel (**Ctrl+Shift+X**) and install:

Extension	Purpose
Python (Microsoft)	Language support, IntelliSense
Pylance	Fast type checking and autocompletion
Jupyter	Run <code>.ipynb</code> notebooks inside VSCode
Rainbow CSV	Color-coded CSV preview

1.4 Configure the Python Interpreter

Press **Ctrl+Shift+P** → type **Python: Select Interpreter** → choose the Python 3.8+ environment where PySpark is installed.

1.5 Create the Main Script

Create `src/lab5_pipeline.py`. Place this header at the top — fill in your name and ID before writing any code:

```
# CS4074 Lab 5 -- NYC Taxi Analytics Pipeline
# Student Name: -----
# Student ID:   -----
# Date:        -----

import os, time
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
from pyspark.sql.window import Window
```

Always run your script from the terminal inside VSCode (**Ctrl+`**) using:

```
1 cd ~/cs4074_lab5
2 python src/lab5_pipeline.py
```

This ensures the HDFS environment variables are correctly inherited.

Part 2

Download Dataset & Load into HDFS (20 min)

You will use the NYC Yellow Taxi Trip Data published by the NYC Taxi & Limousine Commission (TLC) — a real-world public dataset with ≈ 3 million trips recorded in January 2024.

Real taxi data has the same challenges as enterprise data: null values, negative fares from reversals, outliers, multi-key joins, temporal features, and geospatial IDs. Every cleanup step you do here mirrors production data engineering.

2.1 Download the Files

► Terminal - run from ~/cs4074_lab5/data/

```
1 cd ~/cs4074_lab5/data
2
3 # Main trip data (~51 MB Parquet)
4 wget https://d37ci6vzurychx.cloudfront.net/trip-data/
   yellow_tripdata_2024-01.parquet
5
6 # Zone lookup table (~12 KB CSV)
7 wget https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.
   csv
8
9 # Verify both files
10 ls -lh
```

Task 2.1 — Record File Information

After downloading, fill in:

- Parquet file size on disk: 47.65MB
- CSV file size on disk: 12.04KB
- Parquet file format: columnar / row-based (circle one) **Columnar**
- Does the Parquet file contain an embedded schema? Yes / No (circle one) **Yes**

2.2 Create HDFS Directory Structure

► Terminal

```
1 hdfs dfs -mkdir -p /cs4074/lab5/data
2 hdfs dfs -mkdir -p /cs4074/lab5/output
3
4 # Verify directories
5 hdfs dfs -ls /cs4074/
```

2.3 Upload Data to HDFS

```

▶ Terminal

1 # Upload Parquet file (may take 15-30 seconds)
2 hdfs dfs -put ~/cs4074_lab5/data/yellow_tripdata_2024-01.parquet \
3   /cs4074/lab5/data/
4
5 # Upload zone lookup CSV
6 hdfs dfs -put ~/cs4074_lab5/data/taxi_zone_lookup.csv \
7   /cs4074/lab5/data/
8
9 # Verify upload
10 hdfs dfs -ls /cs4074/lab5/data/
11 hdfs dfs -du -h /cs4074/lab5/data/

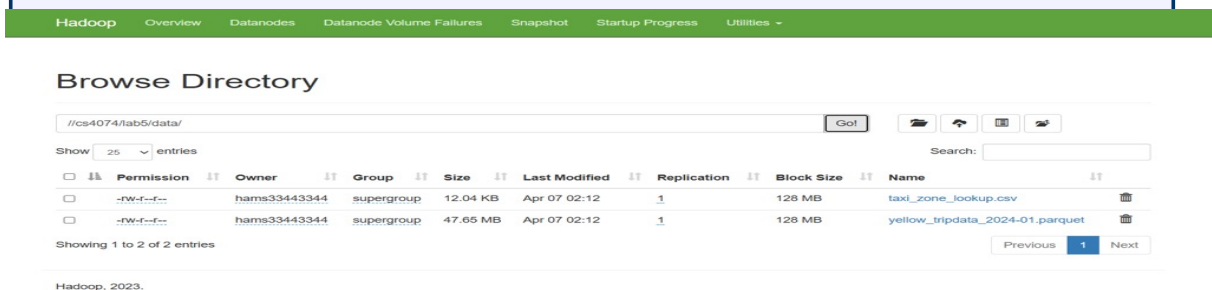
```

2.4 Verify via Hadoop Web UI

Open <http://localhost:9870> → Utilities → Browse File System → navigate to /cs4074/lab5/data/.

Task 2.2 — HDFS Verification Screenshot

Take a screenshot of the Hadoop Web UI showing both files in HDFS.



Run the command below and answer the questions:

```
1 hdfs fsck /cs4074/lab5/data/yellow_tripdata_2024-01.parquet -files -blocks
```

- How many HDFS blocks does the Parquet file occupy? 1 HDFS block
- Each HDFS block is 128 MB. Since 51 MB < 128 MB, the file occupies $\lceil 51 \div 128 \rceil = 1$ block(s). Does this match **fsck**? Yes it matches the fsck output which is 1 blocks, Each HDFS block is 128 MB the file size is approximately 50MB 50<128. 50/128=1 block
- Predict: how many Spark partitions will be created when we read this file? 1 partition since spark typically creates one partition per HDFS block

Part 3

Create the SparkSession (10 min)

The `SparkSession` IS the Application Driver. When you call `getOrCreate()`, a JVM process starts, negotiates with the cluster manager for executor containers, and initialises the Catalyst SQL engine. In PySpark, your Python code connects to this JVM via the Py4J bridge.

Add the following to `src/lab5_pipeline.py`:

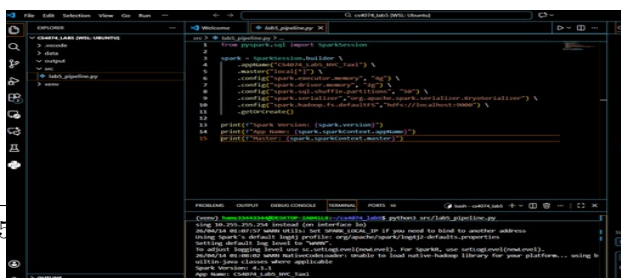
```
1 # Part 3: SparkSession
2
3 spark = SparkSession.builder \
4     .appName('CS4074_Lab5_NYC_Taxi') \
5     .master('local[*]') \
6     .config('spark.executor.memory', '4g') \
7     .config('spark.driver.memory', '2g') \
8     .config('spark.sql.shuffle.partitions', '50') \
9     .config('spark.serializer',
10            'org.apache.spark.serializer.KryoSerializer') \
11     .config('spark.hadoop.fs.defaultFS',
12            'hdfs://localhost:9000') \
13     .getOrCreate()
14
15 print(f"Spark Version : {spark.version}")
16 print(f"App Name      : {spark.sparkContext.appName}")
17 print(f"Master        : {spark.sparkContext.master}")
18 print("Open Spark UI : http://localhost:4040")
```

Configuration Parameter	What It Does	Theory Link
<code>master('local[*]')</code>	Run locally using ALL CPU cores	Cluster Manager
<code>spark.executor.memory = 4g</code>	4 GB per executor; usable = 3796 MB	On-Heap Memory
<code>spark.driver.memory = 2g</code>	RAM for the Application Driver JVM	Driver Process
<code>spark.sql.shuffle.partitions=</code>	Partitions after each shuffle	Wide Transforms
<code>KryoSerializer</code>	Faster object serialisation	Data Transfer
<code>fs.defaultFS = hdfs://...</code>	Points Spark to local HDFS NameNode	HDFS Integration

Task 3.1 — Memory Calculation

Given `spark.executor.memory = 4g`, complete the memory map from the lecture:

- Reserved Memory: 4 GB = 4096 MB MB (always fixed)
- Usable Memory: $4096 - \text{Reserved} = \underline{4096 - 300 = 3796 \text{ MB}}$ MB
- Unified Memory Pool (60% of usable): 2277.6 MB MB
- User Memory (40% of usable, for your Python objects): 1518.4 MB



Part 4

Data Ingestion from HDFS (15 min)

Spark reads HDFS files into partitions. For Parquet, the number of partitions is determined by the number of row groups in the file. For CSV, it follows the HDFS block boundaries (128 MB each). Each partition becomes exactly one task in the first stage.

4.1 Read the Parquet Trip Data

```
1 # Part 4: Data Ingestion
2
3 HDFS_BASE = 'hdfs://localhost
4
5 # Parquet: schema is embedded
6 df_trips = spark.read.parquet
7   (parquet')
8
9 df_trips.printSchema()
10 print(f"Total rows : {df_trip
11 print(f"Partitions : {df_trip
12 df_trips.show(5, truncate=False)
```

only showing top 5 rows

locationid	borough	zone	service_zone
14	Manhattan	Midtown	Manhattan
15	Manhattan	Midtown	Manhattan
16	Manhattan	Midtown	Manhattan
17	Manhattan	Midtown	Manhattan
18	Manhattan	Midtown	Manhattan

only showing top 10 rows

locationid	borough	zone	service_zone
19	Manhattan	Midtown	Manhattan
20	Manhattan	Midtown	Manhattan
21	Manhattan	Midtown	Manhattan
22	Manhattan	Midtown	Manhattan
23	Manhattan	Midtown	Manhattan

```
1 # Parquet: schema is embedded
2 df_trips = spark.read.parquet
3   (parquet')
4
5 df_trips.printSchema()
6 print(f"Total rows : {df_trip
7 print(f"Partitions : {df_trip
8 df_trips.show(5, truncate=False)
```

Task 4.1 — Partition Verification

- Actual partition count: 4
- Does this match your prediction from Reflection 2.1? Yes / No **No**
- Explain why Parquet may produce a different partition count than raw CSV:

In Reflection 2.1, the prediction was based on the HDFS block count (1 block). However, Parquet files are columnar and internally divided into row groups and metadata sections. Spark can split Parquet files into multiple tasks for parallel processing, so the actual number of partitions can be greater than the number of HDFS blocks. This improves performance by allowing multiple cores to process the file simultaneously.

4.2 Read the Zone Lookup CSV with Explicit Schema

`inferSchema=True` forces Spark to read the file *twice*: once to detect types, once to load data. On a 100 GB file this doubles your read time. Always define schemas explicitly in production.

```
1 # Explicit schema: one read pass only
2 zone_schema = StructType([
3     StructField('LocationID', IntegerType(), True),
4     StructField('Borough', StringType(), True),
5     StructField('Zone', StringType(), True),
6     StructField('service_zone', StringType(), True),
7 ])
8
9 df_zones = spark.read \
10     .option('header', 'true') \
11     .schema(zone_schema) \
12     .csv(f'{HDFS_BASE}/data/taxi_zone_lookup.csv')
13
14 df_zones.printSchema()
```

```
15 df_zones.show(10)
```

4.3 Explore Null Values

```
1 # Count nulls in every column
2 null_counts = df_trips.select([
3     sum(col(c).isNull().cast('int')).alias(c)
4     for c in df_trips.columns
5 ])
6 null_counts.show()
```

Task 4.2 — Data Quality Report

List the columns that contain null values and their counts:

Column Name	Null Count
Column Name	Null Count
passenger_count	140162
RatecodeID	140162
store_and_fwd_flag	140162
congestion_surcharge	140162
Airport_fee	140162

Part 5

DataFrame Operations (45 min)

Every operation below is either a **transformation** (lazy — added to the DAG) or an **action** (triggers computation). Before each task, classify the operation. Verify with `explain()` where indicated.

For each subsection, write N (Narrow) or W (Wide) in the box before running the code.

5.1 Filtering — Remove Invalid Trips

☐ Type: Narrow / Wide (circle one)

```

1 # Task 5.1: Remove data quality issues

2 df_valid = df_trips.filter(
3     (col('fare_amount') > 0) &
4     (col('trip_distance') > 0) &
5     (col('total_amount') > 0) &
6     (col('passenger_count') > 0)
7 )
8
9 original_count = df_trips.count()
10 valid_count    = df_valid.count()
11 removed        = original_count - valid_count
12
13 print(f"Original rows : {original_count:,}")
14 print(f"Valid rows    : {valid_count:,}")
15 print(f"Removed rows   : {removed:,} ({removed/original_count*100:.1f}%)")

```

When combining conditions with `&` (AND) or `|` (OR), wrap each condition in parentheses. Python's operator precedence evaluates `&` before `>`, causing silent logic errors otherwise.

```

# WRONG -- Python evaluates & before >
df.filter(col("a") > 3 & col("b") < 5)

# CORRECT
df.filter((col("a") > 3) & (col("b") < 5))

```

5.2 Adding Computed Columns with `withColumn`

☐ Type: Narrow / Wide (circle one)

```

1 # Task 5.2: Enrich with new columns

2 df_enriched = df_valid \
3     .withColumn('tip_pct',
4                 round(col('tip_amount') / col('fare_amount') * 100, 2)) \
5     .withColumn('trip_category',
6                 when(col('trip_distance') < 1, 'Short')
7                 .when(col('trip_distance') < 5, 'Medium')
8                 .when(col('trip_distance') < 15, 'Long')
9                 .otherwise('Very Long')) \

```

```

10 .withColumn('pickup_hour', hour(col('tpep_pickup_datetime'))) \
11 .withColumn('is_weekend',
12             dayofweek(col('tpep_pickup_datetime')).isin([1, 7])) \
13 .withColumn('is_airport_trip',
14             (col('PULocationID').isin(132, 138)) |
15             (col('DOLocationID').isin(132, 138)))
16
17 df_enriched.select(
18     'trip_distance', 'trip_category', 'fare_amount',
19     'tip_pct', 'pickup_hour', 'is_weekend', 'is_airport_trip'
20 ).show(10)

```

Task 5.2 — Column Design

- Location IDs 132 and 138 correspond to which airports? 132 = JFK Airport, 138 = LaGuardia Airport
- The `dayofweek()` function returns 1 for Sunday and 7 for Saturday. Write the condition to capture Monday–Friday instead: `dayofweek(col("tpep_pickup_datetime")).between(2,6)`

This captures Monday (2) through Friday (6).

5.3 Aggregations with `groupBy`

☐ **Type:** Narrow / Wide (circle one)

```

1 # Task 5.3: Multi-aggregation
2
3 # Revenue analysis by trip category
4 df_enriched.groupBy('trip_category').agg(
5     count('*').alias('num_trips'),
6     round(sum('total_amount'), 2).alias('total_revenue'),
7     round(avg('fare_amount'), 2).alias('avg_fare'),
8     round(avg('tip_pct'), 2).alias('avg_tip_pct'),
9     round(avg('trip_distance'), 2).alias('avg_distance'),
10 ).orderBy(desc('total_revenue')).show()
11
12 # Peak hour analysis
13 df_enriched.groupBy('pickup_hour').agg(
14     count('*').alias('num_trips'),
15     round(avg('fare_amount'), 2).alias('avg_fare'),
16 ).orderBy('pickup_hour').show(24)

```

`groupBy('trip_category')` must send all rows with the same category to the same partition using hash partitioning: `partition = hash(key) mod N`. In the Spark UI you will see an **Exchange** node and the stage splits into two (one before, one after the shuffle).

Task 5.3 — Spark UI Check

After running the `groupBy`, open <http://localhost:4040> → Stages tab.

- How many stages does the `groupBy` aggregation create? The groupBy aggregation creates 2 stages: one stage before the shuffle and one stage after the shuffle.
- How many tasks appear in the stage AFTER the shuffle? The stage after the shuffle shows 4 tasks in this execution.
- This number should equal `spark.sql.shuffle.partitions`. Does it? 

No.

Although `spark.sql.shuffle.partitions` was set to 50, Adaptive Query Execution (AQE) optimized the plan and reduced the number of shuffle partitions to 4 based on the dataset size and runtime statistics.

The screenshot shows the Databricks Spark UI. The 'Stages for All Jobs' tab is selected, displaying a table of stages. Stage 10 is highlighted, showing a BroadcastHashJoin operation. The 'Skipped Stages' tab is also visible, showing that stages 11 through 15 were skipped.

5.4 Broadcast Join with Zone Lookup

Type: Narrow / Wide (circle one) — *think*

```

1 # Task 5.4: Join trips with zone lookup
2
3 # zones has only 265 rows -- ideal for broadcast join
4 pickup_zones = df_zones \
5     .withColumnRenamed('LocationID', 'PULocationID') \
6     .withColumnRenamed('Borough', 'pickup_borough') \
7     .select('PULocationID', 'pickup_borough', 'pickup_zone')
8
9 df_with_zones = df_enriched.join(
10     broadcast(pickup_zones),
11     on='PULocationID',
12     how='left'
13 )
14
15 # Verify: the join should NOT create an Exchange node for the zones table
16 df_with_zones.explain()
17
18 # Revenue by borough
19 df_with_zones.groupBy('pickup_borough').agg(
20     count('*').alias('num_trips'),
21     round(sum('total_amount'), 2).alias('total_revenue'),
22     round(avg('tip_pct'), 2).alias('avg_tip_pct'),
23 ).orderBy(desc('total_revenue')).show()

```

When one table fits in memory (<10 MB by default), Spark sends (broadcasts) a copy to every executor. No rows of the large table ever move across the network. Look for **BroadcastHashJoin** instead of **SortMergeJoin** in the execution plan.

Task 5.4 — Execution Plan Analysis

Read the `explain()` output of `df_with_zones` and answer:

- Is the join type **BroadcastHashJoin** or **SortMergeJoin**? The join type is BroadcastHashJoin.
- Is there an **Exchange** node for the zones side of the join? Yes / No
- What does this tell you about network traffic for this join?

Since the zones table is broadcast, Spark avoids shuffling the large taxi dataset across the network. This reduces network traffic and improves join performance.

No, there is no shuffle Exchange node for the zones side. Instead, Spark uses BroadcastExchange to send the small lookup table to all executors.

5.5 Handling Null Values

```

1 # Task 5.5: Null handling

2 # How many trips have no matching zone?
3 null_borough = df_with_zones.filter(col('pickup_borough').isNull()).count()
4 print(f"Trips with no matching zone: {null_borough}")

5

6 # Fill nulls with meaningful defaults
7 df_clean = df_with_zones.fillna({
8     'pickup_borough': 'Unknown',
9     'pickup_zone': 'Unknown',
10    'tip_pct': 0.0,
11 })

```

5.6 Sorting and Pivot Table

```

1 # Task 5.6: Sort and Pivot

2

3 # Top 10 most expensive trips
4 df_clean.select(
5     'tpep_pickup_datetime', 'pickup_borough', 'trip_distance',
6     'fare_amount', 'tip_amount', 'total_amount'
7 ).orderBy(desc('total_amount')).show(10, truncate=False)

8

9 # Pivot: average fare by borough x trip_category
10 df_clean.groupBy('pickup_borough') \
11     .pivot('trip_category', ['Short', 'Medium', 'Long', 'Very Long']) \
12     .agg(round(avg('fare_amount'), 2)) \
13     .orderBy('pickup_borough') \
14     .show()

```

Classify every operation in Part 5 as Narrow (N) or Wide (W), and for each Wide operation, identify the shuffle strategy used:

Operation	N / W	Shuffle strategy (if W)
filter(...)	N	
withColumn(...)	N	
groupBy(...)	W	hash partitioning:
join(broadcast(...))	N	no shuffle (broadcast)
fillna(...)	N	
orderBy(...)	W	global sort shuffle
pivot(...)	W	shuffle + aggregation

The left screenshot shows the execution of a filter operation. The code is: `df_clean.select('tpep_pickup_datetime', 'pickup_borough', 'trip_distance', 'fare_amount', 'tip_amount', 'total_amount').orderBy(desc('total_amount')).show(10, truncate=False)`. The output shows the top 10 most expensive trips, with columns: tpep_pickup_datetime, pickup_borough, trip_distance, fare_amount, tip_amount, total_amount. The right screenshot shows the execution of a pivot operation. The code is: `df_clean.groupBy('pickup_borough').pivot('trip_category', ['Short', 'Medium', 'Long', 'Very Long']).agg(round(avg('fare_amount'), 2)).orderBy('pickup_borough').show()`. The output shows the average fare by borough and trip category, with columns: pickup_borough, Short, Medium, Long, Very Long. The data is as follows:

pickup_borough	Short	Medium	Long	Very Long
Manhattan	16.88	11.35	11.35	11.35
Queens	11.35	11.35	11.35	11.35
Brooklyn	11.35	11.35	11.35	11.35
Manhattan	11.35	11.35	11.35	11.35
Queens	11.35	11.35	11.35	11.35
Brooklyn	11.35	11.35	11.35	11.35
Manhattan	11.35	11.35	11.35	11.35
Queens	11.35	11.35	11.35	11.35
Brooklyn	11.35	11.35	11.35	11.35
Manhattan	11.35	11.35	11.35	11.35

Part 6

Spark SQL & Window Functions (20 min)

Both the DataFrame API and Spark SQL pass through the same Catalyst Optimizer pipeline: Unresolved Plan → Resolved Plan → Optimized Plan → Physical Plan. The `explain()` output of equivalent queries is **identical**.

6.1 Register Temporary Views

```
1 # Part 6: Spark SQL
2 df_clean.createOrReplaceTempView('trips')
3 df_zones.createOrReplaceTempView('zones')
4 print("Views registered: trips, zones")
```

6.2 SQL Queries on Real Taxi Data

```
1 # Revenue breakdown by borough and trip category
2 spark.sql("""
3     SELECT
4         pickup_borough,
5         trip_category,
6         COUNT(*)
7         ROUND(SUM(total_amount), 2) AS total_amount,
8         ROUND(AVG(fare_amount), 2) AS avg_fare_amount,
9         ROUND(AVG(tip_pct), 2) AS avg_tip_pct
10    FROM trips
11   WHERE pickup_borough != 'Unknown'
12     AND fare_amount > 0
13   GROUP BY pickup_borough, trip_category
14   ORDER BY pickup_borough, total_revenue DESC
15 """).show(30)
16
17 # Top 10 busiest hours
18 spark.sql("""
19     SELECT
20         pickup_hour,
21         COUNT(*) AS num_trips,
22         ROUND(AVG(fare_amount), 2) AS avg_fare_amount
23    FROM trips
24   GROUP BY pickup_hour
25   ORDER BY num_trips DESC
26   LIMIT 10
27 """).show()
```

```
python3 lab5_pipeline.py
== DataFrame API Plan ==
- Physical Plan -
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[pickup_borough#25], functions=[sum(total_amount#16)])
+- Exchange hashpartitioning(pickup_borough#25, 50), ENSURE_REQUIREMENTS, [plan_id=43]
+- HashAggregate(keys=[pickup_borough#25], functions=[partial_sum(total_amount#16)])
+- Project [total_amount#16, Borough#20 AS pickup_borough#25]
+- BroadcastHashJoin [PULocationID#7], [LocationID#19], LeftOuter, BuildRight, false
+- Project [PULocationID#7, total_amount#16]
+- Filter (((isnotnull(trip_distance#4) AND isnotnull(fare_amount#10)) AND (trip_distance#4 > 0.0)) AND (fare_amount#10 > 5.0))
+- FileScan parquet [trip_distance#4,PULocationID#7,fare_amount#10,total_amount#16] B
atched: true, DataFilters: [isnotnull(trip_distance#4), isnotnull(fare_amount#10), (trip_distance#4 > 0.0), (fare_amount#10 > 5.0)], Format: Parquet, Location: InMemoryFileIndex(1 paths)[hdfs://localhost:9000/cs4074/labs/data/yellow_tripdata_2024-01.parquet], PartitionFilters: [], PushedFilters: [isnotnull(trip_distance), isnotnull(fare_amount), GreaterThan(trip_distance,0.0), GreaterThan(fare_amount,5.0)], ReadSchema: struct<trip_distance:double,PULocationID:int,fare_amount:double,total_amount:double>
+- BroadcastExchange HashedRelationBroadcastMode(List(cast(input[0, int, false] as bigint)), false), [plan_id=38]
+- Filter isnotnull(LocationID#19)
+- FileScan csv [LocationID#19,Borough#20] Batched: false, DataFilters: [isnotnull(LocationID#19)], Format: CSV, Location: InMemoryFileIndex(1 paths)[hdfs://localhost:9000/cs4074/labs/data/taxi_zone_lookup.csv], PartitionFilters: [], PushedFilters: [isnotnull(LocationID)], ReadSchema: struct<LocationID:int,Borough:string>
```

```
== SQL Plan ==
- Physical Plan -
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[pickup_borough#25], functions=[sum(total_amount#16)])
+- Exchange hashpartitioning(pickup_borough#25, 50), ENSURE_REQUIREMENTS, [plan_id=85]
+- HashAggregate(keys=[pickup_borough#25], functions=[partial_sum(total_amount#16)])
+- Project [total_amount#16, Borough#20 AS pickup_borough#25]
+- BroadcastHashJoin [PULocationID#7], [LocationID#19], LeftOuter, BuildRight, false
+- Project [PULocationID#7, total_amount#16]
+- Filter (((isnotnull(trip_distance#4) AND isnotnull(fare_amount#10)) AND (trip_distance#4 > 0.0)) AND (fare_amount#10 > 5.0))
+- FileScan parquet [trip_distance#4,PULocationID#7,fare_amount#10,total_amount#16] B
atched: true, DataFilters: [isnotnull(trip_distance#4), isnotnull(fare_amount#10), (trip_distance#4 > 0.0), (fare_amount#10 > 5.0)], Format: Parquet, Location: InMemoryFileIndex(1 paths)[hdfs://localhost:9000/cs4074/labs/data/yellow_tripdata_2024-01.parquet], PartitionFilters: [], PushedFilters: [isnotnull(trip_distance), isnotnull(fare_amount), GreaterThan(trip_distance,0.0), GreaterThan(fare_amount,5.0)], ReadSchema: struct<trip_distance:double,PULocationID:int,fare_amount:double,total_amount:double>
+- BroadcastExchange HashedRelationBroadcastMode(List(cast(input[0, int, false] as bigint)), false), [plan_id=80]
+- Filter isnotnull(LocationID#19)
+- FileScan csv [LocationID#19,Borough#20] Batched: false, DataFilters: [isnotnull(LocationID#19)], Format: CSV, Location: InMemoryFileIndex(1 paths)[hdfs://localhost:9000/cs4074/labs/data/taxi_zone_lookup.csv], PartitionFilters: [], PushedFilters: [isnotnull(LocationID)], ReadSchema: struct<LocationID:int,Borough:string>
```

6.3 Window Functions — Rank Trips Within Borough

```
1 # Rank top 5 most expensive trips in each borough
2 ranked = spark.sql("""
3     SELECT
4         pickup_borough,
5         pickup_zone,
6         trip_distance,
7         fare_amount,
```

```

8         total_amount,
9         RANK() OVER (
10             PARTITION BY pickup_borough
11             ORDER BY total_amount DESC
12         ) AS rank_in_borough,
13         ROUND(AVG(fare_amount) OVER (
14             PARTITION BY pickup_borough
15             ), 2) AS borough_avg_fare
16     FROM trips
17     WHERE pickup_borough NOT IN ('Unknown', 'EWR')
18         AND fare_amount > 0
19 """
20
21 ranked.filter(col('rank_in_borough') <= 5) \
22     .orderBy('pickup_borough', 'rank_in_borough') \
23     .show(30, truncate=False)

```

Task 6.1 — Same Plan Proof

Run `explain()` on the same aggregation written both ways and compare:

```

# DataFrame API
df_clean.filter(col('fare_amount') > 5) \
    .groupBy('pickup_borough') \
    .agg(sum('total_amount').alias('total')) \
    .explain()

# SQL version
spark.sql("""
    SELECT pickup_borough, SUM(total_amount) AS total
    FROM trips WHERE fare_amount > 5
    GROUP BY pickup_borough
""").explain()

```

Are the two plans identical? Yes / No Explain what this proves about Catalyst:

Yes, the two physical execution plans are identical except for minor internal plan ID numbers. Both the DataFrame API and SQL query produce the same optimized plan:

Filter.BroadcastHashJoin. Project. HashAggregate. Exchange hashpartitioning. This proves that Spark's Catalyst Optimizer converts both SQL and DataFrame API code into the same optimized logical and physical execution plan. Therefore, when the operations are equivalent, performance is the same.

The performance is identical. So when would you choose SQL over the DataFrame API in a real project? Give two practical reasons, and when would you choose the DataFrame API instead?

Since performance is identical, the choice depends on readability, team skills, and project requirements.

I would choose SQL when:

Team members or analysts are already familiar with SQL and can write queries quickly.

Reporting queries with joins, filters, grouping, and aggregations are easier to read and maintain in SQL syntax.

I would choose the DataFrame API when:

The project needs Python logic such as loops, conditions, variables, reusable functions, or dynamic query generation.

The pipeline combines data processing with machine learning or custom transformations inside one Python application.

Part 7

User-Defined Functions (15 min)

Built-in Spark functions run entirely inside the JVM. Python UDFs force row-by-row data serialisation across the JVM↔Python boundary via Py4J. Pandas UDFs use Apache Arrow to send *batches* across this boundary — much faster, but still Python. Rule: built-in ≫ Pandas UDF ≫ Python UDF.

7.1 Python UDF (Slowest)

```

1 # Part 7: UDF Benchmarks
2 from pyspark.sql.functions import udf
3
4 def classify_fare(fare):
5     if fare is None: return 'Unknown'
6     elif fare < 5:   return 'Low'
7     elif fare < 15:  return 'Standard'
8     elif fare < 30:  return 'High'
9     else:           return 'Premium'
10
11 classify_fare_udf = udf(classify_fare, StringType())
12
13 start = time.time()
14 df_clean.withColumn('fare_tier_udf',
15                     classify_fare_udf(col('fare_amount'))).count()
16 udf_time = time.time() - start
17 print(f"Python UDF time : {udf_time:.2f}s")

```

```

=== Benchmark Results ===
Python UDF Time: 7.35 seconds
Pandas UDF Time: 0.88 seconds
Built-in Time: 0.71 seconds
Speedup Pandas vs Python: 8.38 x
Speedup Built-in vs Python: 10.32 x

```

7.2 Built-in when().otherwise() (Fastest)

```

1 start = time.time()
2 df_clean.withColumn('fare_tier_builtin',
3                     when(col('fare_amount') < 5, 'Low')
4                     .when(col('fare_amount') < 15, 'Standard')
5                     .when(col('fare_amount') < 30, 'High')
6                     .otherwise('Premium')
7                     ).count()
8 builtin_time = time.time() - start
9 print(f"Built-in time : {builtin_time:.2f}s")

```

7.3 Pandas UDF (Vectorised — Fast Alternative)

```

1 import pandas as pd
2 from pyspark.sql.functions import pandas_udf
3
4 @pandas_udf(StringType())
5 def classify_fare_fast(fares: pd.Series) -> pd.Series:
6     return fares.apply(
7         lambda f: 'Unknown' if pd.isna(f) else
8                   'Low'      if f < 5 else
9                   'Standard' if f < 15 else
10                   'High'     if f < 30 else 'Premium'
11     )

```

```

12
13 start = time.time()
14 df_clean.withColumn('fare_tier_pandas',
15                     classify_fare_fast(col('fare_amount'))).count()
16 pandas_time = time.time() - start
17 print(f"Pandas UDF time : {pandas_time:.2f}s")

```

7.4 Benchmark Report

```

print(f"\nPython UDF : {udf_time:.2f}s")
print(f"Pandas UDF : {pandas_time:.2f}s")
print(f"Built-in : {builtin_time:.2f}s")
print(f"Built-in is {udf_time/builtin_time:.1f}x faster than Python UDF")

```

Task 7.1 — Benchmark Results

Method	Time (s)	Speedup vs UDF
Python UDF	7.35	1× (baseline)
Pandas UDF	0.88	8.38x
Built-in	0.71	10.32x

Explain in one sentence why Pandas UDF is faster than Python UDF even though both use Python:

Pandas UDF is faster than Python UDF because it processes data in vectorized batches using Arrow, which greatly reduces row-by-row Python serialization overhead.

Part 8

Partitioning in Practice (15 min)

Each partition is processed by exactly one task on one core. The formula: **tasks in a stage** = **number of partitions**. Too few partitions leaves cores idle; too many creates scheduling overhead. Ideal: 2–4 partitions per available CPU core, each containing 100–200 MB of data.

```

1 # Part 8: Partitioning
2
3 # Current state
4 print(f"df_clean partitions: {df_clean.rdd.getNumPartitions()}")
5
6 # Row distribution across partitions
7 df_clean.withColumn('partition_id', spark_partition_id()) \
8     .groupBy('partition_id') \
9     .count() \
10    .orderBy('partition_id') \
11    .show(60)
12
13 # repartition() -- full shuffle, can increase OR decrease
14 df_by_borough = df_clean.repartition(50, 'pickup_borough')
15 print(f"After repartition(50, borough): {df_by_borough.rdd.getNumPartitions()}")
16
17 # coalesce() -- no shuffle, can only decrease, cheap
18 df_small = df_clean.coalesce(4)
19 print(f"After coalesce(4): {df_small.rdd.getNumPartitions()}")

```

```

Current partition count: 4
Core count: 4
Ideal partition count: between 8 and 16
Partitions after coalesce: 4
=== Explain Plan ===
== Physical Plan ==
Coalesce 4

```

Operation	repartition(n)	coalesce(n)
Shuffle?	Yes — full hash shuffle	No — local merge only
Can increase?	Yes	No (decrease only)
Network cost	High	None
Best for	Increasing partitions; even distribution for joins	Reducing files before write

Task 8.1 — Partition Experiment

- Current partition count of df_clean: 4
- Run: `spark.sparkContext.defaultParallelism`. Result: _ (this is your core count)
- Ideal partition count for your machine (2–4× cores): Between 8 and 16 partitions
- After `coalesce(4)`, does `explain()` show an Exchange node? Yes / No Why?

No because `coalesce()` decreases the number of partitions without performing a full shuffle. Spark combines existing partitions directly, so no Exchange node is required.

4

Part 9

Caching & Persistence (10 min)

Executor memory is split into **Execution Memory** (shuffles, sorts, joins) and **Storage Memory** (cached DataFrames). These share a Unified Memory Pool: by default 60% unified / 40% user, with a sliding boundary between execution and storage within the 60%.

```

1 # Part 9: Caching
2 from pyspark import StorageLevel
3
4 df_clean.cache() # = persist()
5
6 # Caching is lazy -- trigger it with an action
7 start = time.time()
8 df_clean.count() # populates
9 cache_build_time = time.time() - start
10 print(f"First access (builds cache): {cache_build_time}")
11 print(f"Is cached: {df_clean.is_cached}")
12
13 # Second access reads from cache
14 start = time.time()
15 df_clean.count()
16 cache_read_time = time.time() - start
17 print(f"Second access (from cache): {cache_read_time}")
18 print(f"Speedup: {cache_build_time/cache_read_time}")

```

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import *
import time
import builtins

spark = SparkSession.builder \
    .appName("CS4074 Lab5_Parts") \
    .master("local[*]") \
    .config("spark.executor.memory", "2g") \
    .config("spark.driver.memory", "1g") \
    .config("spark.sql.shuffle.partitions", "8") \
    .config("spark.hadoop.fs.defaults", "hdfs://localhost:9000") \
    .getOrCreate()

HDFS_BASE = "hdfs://localhost:9000/cs4074/lab5"

df = spark.read.parquet(f"{HDFS_BASE}/data/yellow_tripdata_2024-01.parquet")
# smaller sample
df_clean = df.filter(col("trip_distance" > 0).limit(500000))

df_clean.cache()

start = time.time()
df_clean.count()
build_time = time.time() - start

start = time.time()
df_clean.count()
read_time = time.time() - start

speedup = build_time / read_time

print("==== Caching Results ====")
print("Build time (first count):", builtins.round(build_time, 2), "s")
print("Read time (second count):", builtins.round(read_time, 2), "s")
print("Speedup:", builtins.round(speedup, 2), "x")

input("Open localhost:4040 Storage tab, then press Enter...")
spark.stop()

```

`df.cache()` sets a caching flag — `df.is_cached` will return `True` immediately. **However, the data is not in memory yet.** Spark caches partitions only as they are computed. You must call an action (`count()`, `show()`, etc.) to actually materialise and store them. Until then, the Storage tab in the Spark UI will show 0 bytes.

```

# Release when finished
df_clean.unpersist()
print(f"After unpersist: {df_clean.is_cached}")

```

Task 9.1 — Caching Results

- Build time (first count): 63.98 s s
- Read time (second count): 1.83 s s
- Speedup: 34.92 x x
- Open <http://localhost:4040> → Storage tab. How many MB does `df_clean` occupy in memory? 49.1 MB
- Is the cached size larger or smaller than the Parquet file on HDFS? Explain why:

The cached size is larger than the parquet file on HDFS because parquet stores data in a compressed columnar format, while Spark caching stores data in a deserialized in-memory format, which takes more space but allows faster processing.

```

(venv) hame33461364@DESKTOP-IAM41L8:~/cs4074/lab5$ python3 src/lab5_pipeline.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/04/22 03:09:28 WARN Utils: Your hostname, DESKTOP-IAM41L8, resolves to a loopback address: 127.0.1.1; u
sing 10.255.255.254 instead (on interface lo)
26/04/22 03:09:28 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/04/22 03:10:15 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using b
uilt-in-java classes where applicable

==== Caching Results ====
Build time (first count): 63.98 s
Read time (second count): 1.83 s
Speedup: 34.92 x

Open localhost:4040 Storage tab, then press Enter...

```

The screenshot shows the Databricks Storage tab for a specific RDD. The RDD is named 'CollectLimit 500000 +- *(1) Filter (isNotNull(trip_distance#4) AND (trip_distance#4 > 0.0)) +- *(1) ColumnarToRow +- FileScan parquet [VendorID#0, tpep_pickup_datetime#1, tpep_dropoff_datetime#2, passenger_count#3, trip_distance#4, RatecodeID#5, store_and_fwd_flag#6, PULocationID#7, DOLocationID#8, payment_type#9, fare_amount#10, extra#11, mtaxi_tax#12, tip_amount#13, tolls_amount#14, improvement_surcharge#15, total_amount#16, congestion_surcharge#17, Airport_fee#18] Batched: true, DataFilters: [isNotNull(trip_distance#4), (trip_distance#4 > 0.0)], Format: Parquet, Location: InMemoryFileIndex(1 paths)[hdfs://localhost:9000/cs4074/lab5/data/yellow_tripdata_2024-01.parquet], PartitionFilters: [], PushedFilters: [IsNotNull(trip_distance), GreaterThan(trip_distance,0.0)], ReadSchema: struct<VendorID:int,tpep_pickup_datetime:timestamp_ntz,tpep_dropoff_datetime:timestamp_ntz,passen...

ID	RDD Name	Storage Level	Cached Partitions	Fraction Cached	Size in Memory	Size on Disk
10	CollectLimit 500000 +- *(1) Filter (isNotNull(trip_distance#4) AND (trip_distance#4 > 0.0)) +- *(1) ColumnarToRow +- FileScan parquet [VendorID#0, tpep_pickup_datetime#1, tpep_dropoff_datetime#2, passenger_count#3, trip_distance#4, RatecodeID#5, store_and_fwd_flag#6, PULocationID#7, DOLocationID#8, payment_type#9, fare_amount#10, extra#11, mtaxi_tax#12, tip_amount#13, tolls_amount#14, improvement_surcharge#15, total_amount#16, congestion_surcharge#17, Airport_fee#18] Batched: true, DataFilters: [isNotNull(trip_distance#4), (trip_distance#4 > 0.0)], Format: Parquet, Location: InMemoryFileIndex(1 paths)[hdfs://localhost:9000/cs4074/lab5/data/yellow_tripdata_2024-01.parquet], PartitionFilters: [], PushedFilters: [IsNotNull(trip_distance), GreaterThan(trip_distance,0.0)], ReadSchema: struct<VendorID:int,tpep_pickup_datetime:timestamp_ntz,tpep_dropoff_datetime:timestamp_ntz,passen...	Disk Memory Deserialized 1x Replicated	1	100.00%	49.1 MiB	0.0 B

The screenshot shows the Databricks Spark Jobs tab. It displays a list of completed jobs with the following details:

- User: hams33443344
- Started At: 2026/04/22 03:10:14
- Total Uptime: 5.2 min
- Scheduling Mode: FIFO
- Completed Jobs: 4

Below the summary, there is a table of completed jobs:

Job ID	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
3	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/04/22 03:12:48	0.1 s	1/1 (1 skipped)	1/1 (4 skipped)
2	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/04/22 03:12:45	0.8 s	1/1 (1 skipped)	1/1 (4 skipped)
1	count at NativeMethodAccessorImpl.java:0 count at NativeMethodAccessorImpl.java:0	2026/04/22 03:11:47	43 s	2/2	5/5
0	parquet at NativeMethodAccessorImpl.java:0 parquet at NativeMethodAccessorImpl.java:0	2026/04/22 03:11:24	7 s	1/1	1/1

Part 10

Writing Results to HDFS (15 min)

At the end of Part 9 you called `df_clean.unpersist()`, which removed it from the cache. In Part 10, `borough_summary` is built from `df_clean`, so Spark will trace back through the entire DAG and recompute `df_clean` from scratch (re-reading HDFS, re-filtering, re-joining). This is correct and expected — it demonstrates why you should cache a DataFrame only when it is *used multiple times*, and release the cache once you are finished with it.

`df.write.save()` is an **action** — it triggers the entire DAG from data ingestion through all transformations to final output. Only after this call does Spark execute your pipeline.

```

1 # Part 10: Write to HDFS

2 borough_summary = df_clean.groupBy('pickup_borough', 'trip_category').agg(
3     count('*').alias('num_trips'),
4     round(sum('total_amount'), 2).alias('total_revenue'),
5     round(avg('fare_amount'), 2).alias('avg_fare'),
6     round(avg('tip_pct'), 2).alias('avg_tip_pct'),
7     round(avg('trip_distance'), 2).alias('avg_distance'),
8 ).orderBy('pickup_borough', desc('total_revenue'))
9
10 # Write 1: Parquet with partition-aware subdirectories
11 borough_summary.write \
12     .mode('overwrite') \
13     .partitionBy('pickup_borough') \
14     .parquet(f'{HDFS_BASE}/output/borough_summary_parquet')
15
16 # Write 2: Single CSV file (coalesce before writing)
17 borough_summary.coalesce(1) \
18     .write \
19     .mode('overwrite') \
20     .option('header', 'true') \
21     .csv(f'{HDFS_BASE}/output/borough_summary_csv')

```

► Verify Output Structure in HDFS

```

1 # Verify partition-aware subdirectories
2 hdfs dfs -ls /cs4074/lab5/output/borough_summary_parquet/
3
4 # Expected: one subdirectory per borough
5 # pickup_borough=Bronx/
6 # pickup_borough=Brooklyn/
7 # pickup_borough=Manhattan/
8 # pickup_borough=Queens/
9 # pickup_borough=Staten Island/
10 # pickup_borough=Unknown/
11
12 # Verify CSV output
13 hdfs dfs -ls /cs4074/lab5/output/borough_summary_csv/

```

Write Mode	Behaviour
overwrite	Delete existing data and write fresh. Use in development.
append	Add new data alongside existing files. Use for incremental loads.
ignore	Do nothing if data already exists. Idempotent writes.
error (default)	Throw exception if output path already exists.

Task 10.1 — Partition-Aware Output Verification

- How many subdirectories were created in `borough_summary_parquet`? 259 partition folders (PULocationID=...)
- Why is `partitionBy('pickup_borough')` useful for future queries? 1 _SUCCESS file
259 subdirectories

It organizes data into partitions based on column values, allowing Spark to read only relevant data instead of scanning the entire dataset, improving performance.

- Why did you use `coalesce(1)` before writing the CSV but not the Parquet?
- `coalesce(1)` was used for CSV to produce a single output file for easier reading. It was not used for Parquet because multiple partitions improve performance and allow parallel processing.

```

7   .config("spark.executor.memory", "2g") \
8   .config("spark.driver.memory", "1g") \
9   .config("spark.hadoop.fs.defaultFS", "hdfs://localhost:9000") \
10  .getOrCreate()
11
12  HDFS_BASE = "hdfs://localhost:9000/cs4074/lab5"
13
14  # Read data
15  df = spark.read.parquet(f"{HDFS_BASE}/data/yellow_tripdata_2024-01.parquet")
16
17  # Clean + simple aggregation
18  df_clean = df.filter(col("trip_distance") > 0)
19
20  summary = df_clean.groupBy("PULocationID") \
21  | .agg(sum("total_amount").alias("total"))
22
23  # Write Parquet partitioned
24  summary.write \
25  | .mode("overwrite") \
26  | .partitionBy("PULocationID") \
27  | .parquet(f"{HDFS_BASE}/output/borough_summary_parquet")
28
29  # Write CSV (single file)
30  summary.coalesce(1).write \
31  | .mode("overwrite") \
32  | .option("header", "true") \
33  | .csv(f"{HDFS_BASE}/output/borough_summary_csv")
34
35  print("Write completed!")
36
37  input("Check HDFS folders, then press Enter...")
38  spark.stop()

```

```

hams33443344@DESKTOP-1A041L8:~$ hdfs dfs -ls /cs4074/lab5/output/borough_summary_parquet | wc -l
260
hams33443344@DESKTOP-1A041L8:~$ hdfs dfs -ls /cs4074/lab5/output/borough_summary_csv
Found 2 items
-rw-r--r-- 3 hams33443344 supergroup 0 2026-05-04 09:19 /cs4074/lab5/output/borough_summary_csv/_SUCCESS
-rw-r--r-- 3 hams33443344 supergroup 4723 2026-05-04 09:19 /cs4074/lab5/output/borough_summary_csv/part-00000-ba
efbda4-4f3f-4ae5-b1ee-be36032bb29c-c000.csv
hams33443344@DESKTOP-1A041L8:~$

```

Part 11

Reading Execution Plans (15 min)

Every Spark query goes through: (1) Unresolved Logical Plan → (2) Resolved Logical Plan → (3) Optimised Logical Plan (Catalyst) → (4) Physical Plans with cost model → (5) DAG of stages and tasks. `explain(True)` shows all five plan representations (parsed, analysed, optimised logical, and physical plan).

```

1 # Part 11: Execution Plan Analysis
2
3 # Run the full pipeline plan
4 print("=== FULL PIPELINE PLAN ===")
5 borough_summary.explain(True)
6
7 # Focused: filter + groupBy
8 print("=== FILTER + GROUPBY PLAN ===")
9 df_clean.filter(col('fare_amount') > 3000) \
10     .groupBy('pickup_borough') \
11     .agg(sum('total_amount').alias('total')) \
12     .explain()

```

Read the physical plan **bottom to top** using this reference:

Plan Node	Meaning	Theory Concept
FileScan parquet [col1,col2]	Reads only listed columns — projection pushdown	Catalyst Optimizer
PushedFilters: [GreaterThan]	Filter sent to file scanner — predicate push-down	Catalyst Optimizer
*(1) Filter (...)	First stage: filter in same stage as scan, no shuffle	Narrow transform
*(1) HashAggregate (partial_sum)	Local partial aggregation before shuffle (Combiner)	Combiner optimisation
Exchange hashpartitioning	SHUFFLE — this is the stage boundary	Wide transform
*(2) HashAggregate (final)	Final aggregation after shuffle (Reducer)	Reduce step
BroadcastHashJoin	No Exchange for small table — broadcast join	Broadcast join
Sort [col ASC]	Range-partition sort — triggers an Exchange before it	Wide transform

Task 11.1 — Plan Reading

From the `explain()` output of `borough_summary`, answer:

- How many **Exchange** (shuffle) nodes appear? 1 Exchange node
- Is the zones join a **BroadcastHashJoin** or **SortMergeJoin**? No join is present in the plan
- Does the filter (`fare_amount > 3000`) appear before or after **FileScan**? What does this tell you about predicate pushdown?

The filter is pushed into the FileScan (predicate pushdown). This means Spark filters data while reading the Parquet file, reducing I/O and improving performance.

d) HashAggregate appears twice. Describe what each one does:

First (before Exchange):

First (before Exchange):

Performs partial aggregation within each partition (local sum)

Second (after Exchange):

Second (after Exchange):

Performs final aggregation after shuffling data across partitions

e) Predicted number of stages: 1 + Exchange count = ____ stages. Verify in Spark UI: ____

2 stages

```

    +- FileScan parquet [trip_distance#4 AND (trip_distance#4 > 0.0)]
    4 > 0.0]], Format: Parquet, Location: InMemoryFileIndex(1 paths)[hdfs://localhost:9000/cs4074/
    [], PushedFilters: [IsNotNull(trip_distance), GreaterThan(trip_distance,0.0)], ReadSchema: st

=== Part 11 Explain Plan (High Fare Filter) ===
== Physical Plan ==
*(1) Filter (((isnotnull(trip_distance#4) AND isnotnull(fare_amount#10)) AND (trip_distance#4
+- *(1) ColumnarToRow
    +- FileScan parquet [VendorID#0, tpep_pickup_datetime#1, tpep_dropoff_datetime#2, passenger_co
    cationID#7, DOLocationID#8, payment_type#9L, fare_amount#10, extra#11, mta_tax#12, tip_amount#13, toll
    on_surcharge#17, Airport_fee#18] Batched: true, DataFilters: [isnotnull(trip_distance#4), isnotn
    ushedFilters: [IsNotNull(trip_distance), IsNotNull(fare_amount), GreaterThan(trip_distance,0.0)
    kup_datetime:timestamp_ntz,tpep_dropoff_datetime:timestamp_ntz,passen...

Write completed!
Check HDFS folders, then press Enter...26/05/04 10:33:47 WARN NettyRpcEnv: Ignored failure: java.
from 10.255.255.254:35381 in 10000 milliseconds
26/05/04 10:33:47 WARN Executor: Issue communicating with driver in heartbeater
org.apache.spark.rpc.RpcTimeoutException: Future timed out after [10000 milliseconds]. This timeo
at org.apache.spark.rpc.RpcTimeout.org$apache$spark$rpc$RpcTimeout$$createRpcTimeoutExcept
at org.apache.spark.rpc.RpcTimeout$$anonfun$addMessageIfTimeout$1.applyOrElse(RpcTimeout.s
at org.apache.spark.rpc.RpcTimeout$$anonfun$addMessageIfTimeout$1.applyOrElse(RpcTimeout.s
at scala.runtime.AbstractPartialFunction.apply(AbstractPartialFunction.scala:35)
at org.apache.spark.rpc.RpcTimeout.awaitResult(RpcTimeout.scala:76)
at org.apache.spark.rpc.RpcEndpointRef.askSync(RpcEndpointRef.scala:101)
at org.apache.spark.executor.Executor.reportHeartBeat(Executor.scala:1359)
at org.apache.spark.executor.Executor.$anonfun$heartbeater$1(Executor.scala:356)
at scala.runtime.java8.JFunction0$mcV$sp.apply(JFunction0$mcV$sp.scala:18)
at org.apache.spark.util.Utils$.logUncaughtExceptions(Utils.scala:1941)
at org.apache.spark.Heartbeater$$anon$1.run(Heartbeater.scala:46)
at java.base/java.util.concurrent.Executors$RunnableAdapter.call(Executors.java:539)

```

Deep Work

End-to-End Pipeline Analysis (30 min)

Build the full pipeline below, which combines everything from Parts 3–11 into a single coherent program. Run it, verify in the Spark UI, and answer the analysis questions.

```

1  #           Deep Work: Complete Pipeline

2  from pyspark.sql import SparkSession
3  from pyspark.sql.functions import *
4  from pyspark.sql.types import *
5
6  spark = SparkSession.builder \
7      .appName('CS4074_Lab5_DeepWork') \
8      .master('local[4]') \
9      .config('spark.executor.memory', '4g') \
10     .config('spark.sql.shuffle.partitions', '8') \
11     .config('spark.hadoop.fs.defaultFS', 'hdfs://localhost:9000') \
12     .getOrCreate()
13
14  HDFS_BASE = 'hdfs://localhost:9000/cs4074/lab5'
15
16  # Step 1: Read
17
18  df = spark.read.parquet(f'{HDFS_BASE}/data/yellow_tripdata_2024-01.
19      parquet')
20
21  # Step 2: Filter (narrow)
22
23  df_filtered = df.filter(
24      (col('fare_amount') > 0) & (col('trip_distance') > 0)
25  )
26
27  # Step 3: Enrich (narrow)
28
29  df_enriched = df_filtered \
30      .withColumn('tip_pct', round(col('tip_amount')/col('fare_amount')
31      *100, 2)) \
32      .withColumn('trip_category',
33          when(col('trip_distance') < 5, 'Short')
34          .when(col('trip_distance') < 15, 'Medium')
35          .otherwise('Long')) \
36      .withColumn('pickup_hour', hour(col('tpep_pickup_datetime')))
37
38  # Step 4: Broadcast join (no shuffle for zones)
39
40  zone_schema = StructType([
41      StructField('LocationID', IntegerType(), True),
42      StructField('Borough', StringType(), True),
43      StructField('Zone', StringType(), True),
44      StructField('service_zone', StringType(), True),
45  ])
46  df_zones = spark.read.option('header', 'true').schema(zone_schema) \
47      .csv(f'{HDFS_BASE}/data/taxi_zone_lookup.csv')
48
49  pickup_zones = df_zones \

```

```

44     .withColumnRenamed('LocationID', 'PULocationID') \
45     .withColumnRenamed('Borough', 'pickup_borough') \
46     .select('PULocationID', 'pickup_borough')
47
48 df_joined = df_enriched.join(broadcast(pickup_zones), 'PULocationID', '
    left')
49 df_clean = df_joined.fillna({'pickup_borough': 'Unknown', 'tip_pct':
    0.0})
50
51 # Step 5: Aggregate (wide -- shuffle!)
52
53 summary = df_clean.groupBy('pickup_borough', 'trip_category').agg(
54     count('*').alias('num_trips'),
55     round(sum('total_amount'), 2).alias('total_revenue'),
56     round(avg('fare_amount'), 2).alias('avg_fare'),
57     round(avg('tip_pct'), 2).alias('avg_tip_pct'),
58 ).orderBy(desc('total_revenue'))
59
60 # Step 6: Cache and display
61
62 summary.cache()
63 summary.show(20)
64
65 # Step 7: Write (wide -- triggers full DAG!)
66
67 summary.coalesce(1) \
68     .write.mode('overwrite').option('header', 'true') \
69     .csv(f'{HDFS_BASE}/output/deep_work_summary')
70
71 # Step 8: Plan inspection
72
73 summary.explain(True)
74
75 # Cleanup
76 summary.unpersist()
77 spark.stop()

```

After running the pipeline, open <http://localhost:4040> and answer:

Q1. The pipeline uses `local[4]`. How many cores are available? How does this relate to the ideal number of partitions? What would you change in `shuffle.partitions`?

The pipeline uses `local[4]`, which means 4 CPU cores are available.

A good rule is to use 2–3x the number of cores for shuffle partitions, so the ideal range is 8–12 partitions.

In this pipeline, `spark.sql.shuffle.partitions` is set to 8, which is appropriate for a local setup and provides good parallelism without creating too many small tasks.

Q2. How many **jobs** were created? Open the Jobs tab and list each job alongside the action that triggered it:

[illegible]

```

47         fare_amount107 = fare_amount10, total_amount106, round((tip.amount103 /
48         WHEN Distance >= 1000 THEN (tip.distance * 0.5) THEN Short WHEN (tip.distance < 15.0)
49         WHEN Distance < 0.55 LONG END AS tip_category212)
50     )
51     )
52     )
53     )
54     )
55     )
56     )
57     )
58     )
59     )
60     )
61     )
62     )
63     )
64     )
65     )
66     )
67     )
68     )
69     )
70     )
71     )
72     )
73     )
74     )
75     )
76     )
77     )
78     )
79     )
80     )
81     )
82     )
83     )
84     )
85     )
86     )
87     )
88     )
89     )
90     )
91     )
92     )
93     )
94     )
95     )
96     )
97     )
98     )
99     )
100    )
101    )
102    )
103    )
104    )
105    )
106    )
107    )
108    )
109    )
110    )
111    )
112    )
113    )
114    )
115    )
116    )
117    )
118    )
119    )
120    )
121    )
122    )
123    )
124    )
125    )
126    )
127    )
128    )
129    )
130    )
131    )
132    )
133    )
134    )
135    )
136    )
137    )
138    )
139    )
140    )
141    )
142    )
143    )
144    )
145    )
146    )
147    )
148    )
149    )
150    )
151    )
152    )
153    )
154    )
155    )
156    )
157    )
158    )
159    )
160    )
161    )
162    )
163    )
164    )
165    )
166    )
167    )
168    )
169    )
170    )
171    )
172    )
173    )
174    )
175    )
176    )
177    )
178    )
179    )
180    )
181    )
182    )
183    )
184    )
185    )
186    )
187    )
188    )
189    )
190    )
191    )
192    )
193    )
194    )
195    )
196    )
197    )
198    )
199    )
200    )
201    )
202    )
203    )
204    )
205    )
206    )
207    )
208    )
209    )
210    )
211    )
212    )
213    )
214    )
215    )
216    )
217    )
218    )
219    )
220    )
221    )
222    )
223    )
224    )
225    )
226    )
227    )
228    )
229    )
230    )
231    )
232    )
233    )
234    )
235    )
236    )
237    )
238    )
239    )
240    )
241    )
242    )
243    )
244    )
245    )
246    )
247    )
248    )
249    )
250    )
251    )
252    )
253    )
254    )
255    )
256    )
257    )
258    )
259    )
260    )
261    )
262    )
263    )
264    )
265    )
266    )
267    )
268    )
269    )
270    )
271    )
272    )
273    )
274    )
275    )
276    )
277    )
278    )
279    )
280    )
281    )
282    )
283    )
284    )
285    )
286    )
287    )
288    )
289    )
290    )
291    )
292    )
293    )
294    )
295    )
296    )
297    )
298    )
299    )
300    )
301    )
302    )
303    )
304    )
305    )
306    )
307    )
308    )
309    )
310    )
311    )
312    )
313    )
314    )
315    )
316    )
317    )
318    )
319    )
320    )
321    )
322    )
323    )
324    )
325    )
326    )
327    )
328    )
329    )
330    )
331    )
332    )
333    )
334    )
335    )
336    )
337    )
338    )
339    )
340    )
341    )
342    )
343    )
344    )
345    )
346    )
347    )
348    )
349    )
350    )
351    )
352    )
353    )
354    )
355    )
356    )
357    )
358    )
359    )
360    )
361    )
362    )
363    )
364    )
365    )
366    )
367    )
368    )
369    )
370    )
371    )
372    )
373    )
374    )
375    )
376    )
377    )
378    )
379    )
380    )
381    )
382    )
383    )
384    )
385    )
386    )
387    )
388    )
389    )
390    )
391    )
392    )
393    )
394    )
395    )
396    )
397    )
398    )
399    )
400    )
401    )
402    )
403    )
404    )
405    )
406    )
407    )
408    )
409    )
410    )
411    )
412    )
413    )
414    )
415    )
416    )
417    )
418    )
419    )
420    )
421    )
422    )
423    )
424    )
425    )
426    )
427    )
428    )
429    )
430    )
431    )
432    )
433    )
434    )
435    )
436    )
437    )
438    )
439    )
440    )
441    )
442    )
443    )
444    )
445    )
446    )
447    )
448    )
449    )
450    )
451    )
452    )
453    )
454    )
455    )
456    )
457    )
458    )
459    )
460    )
461    )
462    )
463    )
464    )
465    )
466    )
467    )
468    )
469    )
470    )
471    )
472    )
473    )
474    )
475    )
476    )
477    )
478    )
479    )
480    )
481    )
482    )
483    )
484    )
485    )
486    )
487    )
488    )
489    )
490    )
491    )
492    )
493    )
494    )
495    )
496    )
497    )
498    )
499    )
500    )
501    )
502    )
503    )
504    )
505    )
506    )
507    )
508    )
509    )
510    )
511    )
512    )
513    )
514    )
515    )
516    )
517    )
518    )
519    )
520    )
521    )
522    )
523    )
524    )
525    )
526    )
527    )
528    )
529    )
530    )
531    )
532    )
533    )
534    )
535    )
536    )
537    )
538    )
539    )
540    )
541    )
542    )
543    )
544    )
545    )
546    )
547    )
548    )
549    )
550    )
551    )
552    )
553    )
554    )
555    )
556    )
557    )
558    )
559    )
560    )
561    )
562    )
563    )
564    )
565    )
566    )
567    )
568    )
569    )
570    )
571    )
572    )
573    )
574    )
575    )
576    )
577    )
578    )
579    )
580    )
581    )
582    )
583    )
584    )
585    )
586    )
587    )
588    )
589    )
590    )
591    )
592    )
593    )
594    )
595    )
596    )
597    )
598    )
599    )
600    )
601    )
602    )
603    )
604    )
605    )
606    )
607    )
608    )
609    )
610    )
611    )
612    )
613    )
614    )
615    )
616    )
617    )
618    )
619    )
620    )
621    )
622    )
623    )
624    )
625    )
626    )
627    )
628    )
629    )
630    )
631    )
632    )
633    )
634    )
635    )
636    )
637    )
638    )
639    )
640    )
641    )
642    )
643    )
644    )
645    )
646    )
647    )
648    )
649    )
650    )
651    )
652    )
653    )
654    )
655    )
656    )
657    )
658    )
659    )
660    )
661    )
662    )
663    )
664    )
665    )
666    )
667    )
668    )
669    )
670    )
671    )
672    )
673    )
674    )
675    )
676    )
677    )
678    )
679    )
680    )
681    )
682    )
683    )
684    )
685    )
686    )
687    )
688    )
689    )
690    )
691    )
692    )
693    )
694    )
695    )
696    )
697    )
698    )
699    )
700    )
701    )
702    )
703    )
704    )
705    )
706    )
707    )
708    )
709    )
710    )
711    )
712    )
713    )
714    )
715    )
716    )
717    )
718    )
719    )
720    )
721    )
722    )
723    )
724    )
7
```

Job ID	Action that triggered it	Stages in job
0	summary.show().	~2–3 stages
1	summary.write().	~3–4 stages
2		

Note: This pipeline has two explicit actions (**show** and **write**), so expect 2–3 jobs. Job IDs are assigned sequentially by Spark. Check the Jobs tab and fill in only the jobs that actually appear.

$$\text{Stages} = \underbrace{1}_{\text{base}} + \underbrace{\quad}_{\text{groupBy shuffles}} + \underbrace{\quad}_{\text{orderBy shuffle}} = \text{total stages}$$

Q4. The pipeline ran `shuffle.partitions=8` instead of the default 200. How did this affect the number of output files in HDFS? Would you use 200 on a production cluster with 50 nodes? Explain.

Setting `shuffle.partitions = 8` reduces the number of partitions created during shuffle

prediction matches.

However, in a production cluster with around 50 nodes, using 200 or more partitions would be appropriate to fully utilize cluster resources and improve parallelism.

Predicate Pushdown

1. Filters such as `(fare_amount > 0)` and `(trip_distance > 0)` are pushed down to the Parquet file scan. This reduces disk I/O by reading only relevant data.

Broadcast Hash Join

- The zones lookup table is broadcast to all worker nodes.
- This avoids a shuffle and reduces network traffic during the join.

Partial Aggregation (HashAggregate twice)

Spark performs partial aggregation before the shuffle and final aggregation after the shuffle.

3. This reduces the amount of data transferred across the network.

Effat University

Deliverables & Grading

Submit the following on the course portal. File name format: CS4074_Lab5_YourName_YourID.zip

#	Item	What to Submit	Marks
1	Python Script	src/lab5_pipeline.py — complete, all parts, with your name header	15
2	Terminal Output	Screenshot showing all <code>print()</code> outputs from the full run	10
3	Spark UI — Jobs	Screenshot of Jobs tab (multiple jobs visible)	10
4	Spark UI — Stages	Screenshot of Stages tab (all pipeline stages)	10
5	Spark UI — Storage	Screenshot of Storage tab while <code>df_clean</code> is cached	5
6	HDFS Output	Screenshot of <code>hdfs dfs -ls /cs4074/lab5/output/</code>	5
7	Task Answers	All filled-in task boxes and reflection questions in this sheet	30
8	Deep Work	Complete answers to Q1–Q5 + Spark UI screenshots	15
Total			100

- File name format: CS4074_Lab5_YourName_YourID.pdf
- Deadline: **One week from today's lab session.**
- Late submissions: **–5 marks per day.**
- Do NOT submit code that was not executed — every `print()` must appear in the terminal screenshot.

```

1 # Stop Spark (already done in script if you called spark.stop())
2 # Stop HDFS when you are done with the lab
3 stop-dfs.sh
4
5 # Verify all daemons stopped
6 jps

```

End of Lab 5

Next Lab: Big Data Analytics with Spark ML